

BAN404 — Machine Learning | Exam Cheat-Sheet

1. FUNDAMENTALS & BIAS-VARIANCE TRADEOFF

1. Info

- $Y = f(X) + \epsilon \rightarrow$ estimate f . Parametric: assume shape (linear etc.), few params, biased if wrong. Non-parametric: no shape (KNN, trees), flexible, high variance.
- $MSE(test) = Bias(\hat{f})^2 + Var(\hat{f}) + Var(\epsilon)$. Goal: minimise test MSE via cross-validation.
- Bias: systematic error (too simple model). Variance: sensitivity to training data (too complex model).
- High $\alpha \rightarrow$ simpler model $\rightarrow$ higher bias, lower variance. Low $\alpha \rightarrow$ complex $\rightarrow$ lower bias, higher variance.
- $R^2 = 1 - RSS/TSS = (TSS - RSS)/TSS$. $TSS = \sum (y - \bar{y})^2$. $MAD = \text{mean}(|y - \hat{y}|)$ — robust to outliers.

2. Evaluering

- Validation set (50/50): enkel, men variabel. k-fold CV: split i k deler, roter valideringsdel, snitt MSE.
- LOO (Leave-One-Out): $k=n$. Minst bias, tregest. Foretrekkes for små datasett.
- Aldri evaluer endelig modell mer enn én gang på testsettet. Bruk treningsdata + CV for modellvalg.

2. DATA PREPARATION & DESCRIPTIVE STATISTICS

1. Info — deskriptiv statistikk for eksamen

- Kontinuerlig Y + kontinuerlig $X \rightarrow$ scatter. Kontinuerlig Y + kategorisk $X \rightarrow$ boxplot.
- Kategorisk Y + kontinuerlig $X \rightarrow$ boxplot. Kategorisk Y + kategorisk $X \rightarrow$ krysstabell (normalize='columns').

2. Manuelt

```
# Korrelasjonsmatrise
df.corr()
# Krysstabell normalisert
pd.crosstab(y, x, normalize='columns')
# Boxplot
df.boxplot('y', by='cat')
# Scatter
plt.scatter(x, y)
```

3. Med pakker (forelesers train/test split)

```
np.random.seed(123)
n = len(df)
train_idx = np.random.choice(n, size=n//2, replace=False)
test_idx = np.setdiff1d(np.arange(n), train_idx) # forelesers metode
train = df.iloc[train_idx]; test = df.iloc[test_idx]

# Alternativ sklearn:
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, random_state=1)
```

4. Annet

```
# Kategorisk y
auto['high'] = (auto['mpg'] > auto['mpg'].median()).astype(int)
y_train = (train['Direction'] == 'Up').astype(int)
# Dummies
X = pd.get_dummies(X_raw, drop_first=True).astype(float).fillna(0)
```

3. OLS — LINEAR REGRESSION

1. Info

- $\beta = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ minimerer $RSS = \sum (y_i - \hat{y}_i)^2$
- $\beta = (X^T X)^{-1} X^T y$ (analytisk løsning). p-verdi tester $H_0: \beta_j = 0$. KI: usikkerhet rundt $f(X)$. PI: usikkerhet rundt Y (bredere).
- Overfitting ved mange variabler/lite data $\rightarrow$ bruk Ridge/LASSO.

2. Manuelt

```
# Analytisk OLS uten pakker
X_c = np.column_stack([np.ones(n), X]) # add intercept
b = np.linalg.solve(X_c.T @ X_c, X_c.T @ y)
yhat = X_c @ b
RSS = np.sum((y - yhat)**2)
TSS = np.sum((y - y.mean())**2)
R2 = 1 - RSS/TSS
```

3. Med pakker (statsmodels — forelesers metode)

```
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import ModelSpec as MS, summarize

Auto = load_data('Auto')
design = MS(['horsepower']).fit(Auto)
X = design.transform(Auto)
y = Auto['mpg']
res = sm.OLS(y, X).fit()
summarize(res) # koeffisienter + p-verdier

# Prediksjon
X_new = design.transform(pd.DataFrame({'horsepower': [98]}))
res.predict(X_new) # punktprediksjon
res.get_prediction(X_new).conf_int(alpha=0.05) # KI
res.get_prediction(X_new).conf_int(obs=True, alpha=0.05) # PI
```

4. Annet — diagnostikk

```
fig, axs = plt.subplots(1, 2, figsize=(10,4))
axs[0].scatter(res.fittedvalues, res.resid)
axs[0].axhline(0, color='black')
sm.qqplot(res.resid, line='45', ax=axs[1])
```

4. RIDGE REGRESSION

1. Info

```
min RSS +  $\alpha \cdot \sum \beta_k^2$  (L2-straft – krymper koeffisienter mot 0, aldri eksakt 0)
```

- $\alpha=0 \rightarrow$ OLS. $\alpha \rightarrow \infty \rightarrow$ alle $\beta \rightarrow 0$. Høy α : mer bias, lavere varians $\rightarrow$ bedre test-MSE ved støy.
- sklearn `fit_intercept=True` (default) demener automatisk — ikke nødvendig å demene manuelt.
- RidgeCV: velger optimal α via k-fold CV på treningsdata. Standardiser X for sammenlignbare koeff.

2. Manuelt (forelesers g()- og loo()-funksjon)

```
# Analytisk: b = (X'X +  $\alpha \cdot I$ )^{-1} X'y (med demeaning)
b0 = y.mean(); yd = y - b0
Xd = X - X.mean(axis=0)
p = Xd.shape[1]
def ridge_coef(alpha, Xd, yd):
    A = Xd.T @ Xd + alpha * np.eye(p)
    return np.linalg.solve(A, Xd.T @ yd)

b1 = ridge_coef(1000, Xd, yd)
ypred = b0 + Xd @ b1
MSE = np.mean((yd - Xd @ b1)**2)

# Med scipy (slik forelesers g() gjør det):
from scipy.optimize import minimize
def f(b1, X, y, la):
    return np.sum((y - X @ b1)**2) + la * np.sum(b1**2)
opt = minimize(f, np.zeros(p), args=(Xd, yd, 1000))

# LOO for Ridge (forelesers loo()-funksjon):
def loo_ridge(alpha, X, y):
    n, q = X.shape
    b0 = y.mean(); yd = y - b0
    Xd = X - X.mean(axis=0)
    ydpred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        b1 = ridge_coef(alpha, Xd[idx], yd[idx])
        ydpred[i] = Xd[i] @ b1
    return np.mean((yd - ydpred)**2)

alphas = np.logspace(-2, 6, 50)
loo_mse = [loo_ridge(a, X, y) for a in alphas]
best_alpha = alphas[np.argmin(loo_mse)]
```

3. Med pakker (sklearn — forelesers ex611.py)

```
import sklearn.linear_model as skl

alphas = np.logspace(-4, 4, 200)
ridge_cv = skl.RidgeCV(alphas=alphas, cv=5)
ridge_cv.fit(Xtrain, ytrain)
print('Optimal alpha:', ridge_cv.alpha_)

ridgemin = skl.Ridge(alpha=ridge_cv.alpha_)
ridgemin.fit(Xtrain, ytrain)
pred = ridgemin.predict(Xtest)
MSE = ((ytest - pred)**2).mean()

# Koeffisienter inkl. intercept:
ridge_coef = np.concatenate([[ridgemin.intercept_], ridgemin.coef_])
```

5. LASSO REGRESSION

1. Info

```
min RSS +  $\alpha \cdot \sum |\beta_k|$  (L1-straft – kan sette  $\beta_k = 0$  eksakt  $\rightarrow$  variabelseleksjon)
```

- L1 har konstant marginalkostnad: $|\beta|$ koster like mye fra 0.01 $\rightarrow$ 0 som 2 $\rightarrow$ 1.99. RSS-gevinst lønner ikke $\rightarrow \beta=0$.
- Ridge: β^2 har synkende marginalkostnad nær 0 $\rightarrow$ Ridge lar aldri β bli eksakt 0.
- Bruk LASSO når du mistenker at bare noen få prediktorer er relevante.

2. Manuelt

```
# LASSO har ingen analytisk lukket form – brukes via scipy eller sklearn.
# Manuell koordinat-nedstigning (konsept):
# For hver  $\beta_j$ : soft-threshold:  $\text{sign}(p_j) \cdot \max(|p_j| - \alpha/2, 0)$ 
# der  $p_j = X_j'(y - X_{-j}\beta_{-j})$ 
# I praksis: alltid bruk sklearn.linear_model.Lasso
```

3. Med pakker (sklearn — forelesers metode)

```
alphas = np.logspace(-4, 4, 200)
lasso_cv = skl.LassoCV(alphas=alphas, cv=5)
lasso_cv.fit(Xtrain, ytrain)

lasso = skl.Lasso(alpha=lasso_cv.alpha_)
lasso.fit(Xtrain, ytrain)
pred = lasso.predict(Xtest)
MSE = ((ytest - pred)**2).mean()

# Koeffisienter (nullede variabler):
lasso_coef = np.concatenate([[lasso.intercept_], lasso.coef_])
pd.DataFrame({'OLS':ols_coef, 'Ridge':ridge_coef, 'Lasso':lasso_coef})
```

4. Annet — med StandardScaler (ames.py)

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler(with_mean=True, with_std=True)
X_tr_s = scaler.fit_transform(Xtrain) # fit KUN på train
X_te_s = scaler.transform(Xtest)
lasso_cv.fit(X_tr_s, ytrain)
```

6. BOOTSTRAP

1. Info

- Simulerer samplingsfordeling uten ny data. Trekk n obs med tilbakelegging B ganger → beregn statistikk.
- Gir B verdier av statistikken → dette ER (en approks. av) samplingsfordelingen.
- 95% KI: persentilmetoden → [2.5%, 97.5%] av bootstrap-verdiene.
- ddof=1 i np.var → S^2 = stikkprøvevarians (deler på n-1, upartisk estimat).

2. Manuelt

```
# En bootstrap-iterasjon for hånd:
idx    = np.random.choice(n, n, replace=True) # med tilbakelegging
y_boot = y[idx]
s2_boot = np.sum((y_boot - y_boot.mean())**2) / (n - 1) # S² manuelt
# Gjenta B ganger og samle s2_boot-verdiene
```

3. Med pakker (forelesers metode)

```
np.random.seed(42)
n = len(y); B = 1000
boot_vars = np.array([
    np.var(y[np.random.choice(n, n, replace=True)]), ddof=1)
    for _ in range(B)
])
ci = np.percentile(boot_vars, [2.5, 97.5])
plt.hist(boot_vars, bins=40)
plt.axvline(np.var(y, ddof=1), color='black', linestyle='--')
```

4. Annet

- Bootstrap av regresjonskoeffisienter: erstatt y[...] med full OLS-fitting på bootstrap-stikkprøven.

7. LOGISTISK REGRESJON

1. Info

$P(Y=1|x) = 1 / (1 + \exp(-(\beta_0 + \beta_1 \cdot x)))$ [sigmoid / logit link]

- Estimeres via maximum likelihood (ikke OLS). $\beta_1 > 0 \rightarrow$ høyere x $\rightarrow$ høyere $P(Y=1)$.
- Log-odds: $\log[P/(1-P)] = \beta_0 + \beta_1 \cdot x \rightarrow$ lineær i log-odds, ikke i P.
- Terskel $P > 0.5 \rightarrow Y=1$ (standard). Kan justeres ved ubalanserte klasser.
- GLM-familien: Binomial()=logistisk, Gaussian()=OLS, Poisson()=telldata.

2. Manuelt

```
# Sigmoid for ett punkt x0:
a0, a1 = m.params[0], m.params[1]
prob0 = 1 / (1 + np.exp(-a0 - a1 * x0))

# Konfusjon uten pakker:
pred = (prob > 0.5).astype(int)
TP = np.sum((pred==1) & (y==1))
TN = np.sum((pred==0) & (y==0))
accuracy = (TP + TN) / len(y)
```

3. Med pakker (statsmodels — forelesers metode)

```
import statsmodels.api as sm

X = sm.add_constant(train[['Lag2']]) # add_constant alltid manuelt!
y = (train['Direction'] == 'Up').astype(int)
m = sm.GLM(y, X, family=sm.families.Binomial()).fit()
print(m.summary())

X_test = sm.add_constant(test[['Lag2']])
prob = m.predict(X_test) # sannsynligheter [0,1]
pred = (prob > 0.5).astype(int)

# Confusion matrix (forelesers metode):
pd.crosstab(y_true.values, pred,
            rownames=['Actual'], colnames=['Predicted'])
np.mean(y_true == pred) # accuracy
```

4. Annet — LOO for logistisk regresjon

```
n = len(auto)
pred = np.zeros(n, dtype=int)
for i in range(n):
    idx = np.arange(n) != i
    X_tr = sm.add_constant(auto.iloc[idx][['Lag2']])
    y_tr = (auto.iloc[idx]['Direction'] == 'Up').astype(int)
    X_te = auto.iloc[[i]][['Lag2']].reindex(columns=X_tr.columns, fill_value=1)
    m = sm.GLM(y_tr, X_tr, family=sm.families.Binomial()).fit()
    pred[i] = int(m.predict(X_te).iloc[0] > 0.5)
```

8. KNN (K-NEAREST NEIGHBOURS)

1. Info

- Regresjon: $\blacksquare(x_0) = (1/K) \cdot \sum y_i$ for K nærmeste naboer.
- Klassifikasjon: $P(Y=1|x_0) = (1/K) \cdot$ antall naboer med $y=1$.
- Lav K $\rightarrow$ overfit (følger støy). Høy K $\rightarrow$ underfit (for glatt). Velg K med LOO-CV.
- Avstand (1D): $|x - x_0|$. Multivariat: Euklidsk avstand (standardiser X!).

2. Manuelt (forelesers metode — 1D)

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0) # avstand
    o = np.argsort(d)[:K] # K nærmeste indekser
    return np.mean(y[o]) # gjennomsnitt (reg.) eller sannsynlighet (klass.)

# LOO for å velge K:
```

```
best_K, best_err = 1, np.inf
for K in range(1, 20):
    err = []
    for i in range(n):
        idx = np.arange(n) != i
        yhat = knn(x[i], x[idx], y[idx], K)
        err.append((y[i] - yhat)**2)
    if np.mean(err) < best_err:
        best_K, best_err = K, np.mean(err)
```

3. Med pakker (sklearn)

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
pred = knn_clf.predict(X_test)
prob = knn_clf.predict_proba(X_test)[:,:1]
```

4. Annet — KNN for GAM (backfitting-smoother)

```
# Brukes av foreleser som smoother i GAM (gam_example.py):
# knn() fra scratch, én prediktor om gangen, på partiell residual
```

9. LDA / QDA / NAIVE BAYES

1. Info

Metode	Antagelse	Bruk
LDA	Lik kovarians alle klasser	Lite data, normalfordeling
QDA	Ulik kovarians per klasse	Mer fleksibel enn LDA
Naive Bayes	Prediktorer uavhengige gitt klasse	Mange prediktorer

- Alle tre estimerer $P(Y=k|X)$ via Bayes' teorem: $P(Y=k|X) \propto P(X|Y=k) \cdot P(Y=k)$.

2. Manuelt — LDA (2 klasser)

```
# μk = klassegjennomsnitt, Σ = poolet kovarians
mu0 = X[y==0].mean(axis=0); mu1 = X[y==1].mean(axis=0)
pi0 = np.mean(y==0); pi1 = np.mean(y==1)
n0 = np.sum(y==0); n1 = np.sum(y==1)
S0 = np.cov(X[y==0].T, ddof=1)
S1 = np.cov(X[y==1].T, ddof=1)
Sp = (n0*S0 + n1*S1) / (n0+n1-2) # poolet kovarians
Spin = np.linalg.inv(Sp)
# Diskriminantfunksjon for ny obs x:
d0 = x @ Spin @ mu0 - 0.5*mu0 @ Spin @ mu0 + np.log(pi0)
d1 = x @ Spin @ mu1 - 0.5*mu1 @ Spin @ mu1 + np.log(pi1)
pred = 1 if d1 > d0 else 0
```

3. Med pakker (sklearn)

```
from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis as LDA,
                                           QuadraticDiscriminantAnalysis as QDA)

from sklearn.naive_bayes import GaussianNB

for Model in [LDA(), QDA(), GaussianNB()]:
    Model.fit(X_train, y_train)
    pred = Model.predict(X_test)
    print(type(Model).__name__, np.mean(pred == y_test))
```

10. POLYNOMREGRESJON & TRAPPEFUNKSJONER

1. Info

Polynom grad K: $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \dots + \beta_K \cdot x^K$ [**K+1** kolonner i X]

- np.vander(x, K+1, increasing=True) gir kolonner [1, x, x², ..., x^K]. Velg K med validation set / CV.
- Trappéfunksjoner: pd.cut deler x i K like store bin → kategorisk variabel → dummyer.

2. Manuelt

```
# Polynomregresjon
K = 7
Xpoly = np.vander(age, K+1, increasing=True) # [1, age, age², ..., age^K]
m = sm.OLS(wage, Xpoly).fit()

# Velg optimal K med validation set MSE:
mse = []
for k in range(1, 11):
    Xtr = np.vander(age_train, k+1, increasing=True)
    m = sm.OLS(wage_train, Xtr).fit()
    Xte = np.vander(age_test, k+1, increasing=True)
    mse.append(((wage_test - m.predict(Xte))**2).mean())
best_k = np.argmin(mse) + 1

# Trappéfunksjoner manuelt:
K = 4
train = train.assign(age_cut=pd.cut(train['age'], K))
test = test.assign(age_cut=pd.cut(test['age'],
    bins=pd.cut(train['age'],K).cat.categories))
Xtr = pd.get_dummies(train['age_cut'], drop_first=True).astype(float)
Xte = pd.get_dummies(test['age_cut'], drop_first=True).astype(float)
Xtr = sm.add_constant(Xtr); Xte = sm.add_constant(Xte)
```

3. Med pakker

```
# sklearn PolynomialFeatures
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3, include_bias=True)
Xpoly = poly.fit_transform(X_train)
```

11. REGRESSION SPLINES

1. Info

Kubisk spline med K knutepkt: basis = {1, x, x², x³, h(x,ξ₁), ..., h(x,ξ_K)}

h(x, ξ) = max((x - ξ)³, 0) [trunkert power basis]

- K knutepunkter → K+4 parametere (kubisk spline). Velg knutepunkter med LOO-CV.
- Natural spline: lineær i halene (reduserer varians). Smoothing spline: automatisk knutepunkt for hvert xi.

2. Manuelt

```
def h(x, xi): return np.maximum((x - xi)**3, 0)

# Kubisk spline med 2 knutepunkter ξ1=3, ξ2=6:
X_sp = np.column_stack([np.ones(n), x, x**2, x**3,
                        h(x, 3.0), h(x, 6.0)])
m = sm.OLS(y, X_sp).fit()

# LOO for å velge knutepunkter:
from itertools import combinations
best_loo = np.inf; best_knots = None
for knots in combinations([1,2,3,4,5,6,7], 2):
    X_sp = np.column_stack([np.ones(n), x, x**2, x**3
                            + [h(x, k) for k in knots]])
    loo_err = []
    for i in range(n):
        idx = np.arange(n) != i
        m = sm.OLS(y[idx], X_sp[idx]).fit()
        loo_err.append((y[i] - m.predict(X_sp[[i]]))**2)
    if np.mean(loo_err) < best_loo:
        best_loo, best_knots = np.mean(loo_err), knots
```

3. Med pakker (ISLP.transforms.BSpline — forelesers metode)

```
from ISLP.transforms import BSpline

# Fit og transform:
bs = BSpline(internal_knots=[3.5, 5.5], degree=3, intercept=True)
Xbs = bs.fit_transform(x.reshape(-1, 1))
m = sm.OLS(y, Xbs).fit()

# OBS: intercept=True nødvendig for å matche manuell basis.
```

12. GAM (GENERALIZED ADDITIVE MODELS)

1. Info

y = b0 + f1(x1) + f2(x2) + ... + fq(xq) [additiv – tolkes per variabel]

- Backfitting: estimer f1 (hold f2,...,fq fast) → residual → estimer f2 → gjenta til konvergens.
- Partiell residual for fk: rk = y - b0 - Σ_{j≠k} fj(xj). Demean etter hvert steg!
- Identifiser ikke-lineære variabler: ΔR² = R²(kvadratisk) - R²(lineær). Stor ΔR² → bruk spline.

2. Manuelt — backfitting med KNN-smoother (forelesers gam_example.py)

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    return np.mean(y[np.argsort(d)[-K:]])

b0 = y.mean(); yd = y - b0; K = 20

# Iterasjon 1:
f1 = b0 + np.array([knn(x0, x1, yd, K) for x0 in x1])
res = y - f1; res -= res.mean()
f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])

# Backfitting (gjenta til konvergens):
for _ in range(10):
    res = y - f2; res -= res.mean()
    f1 = b0 + np.array([knn(x0, x1, res, K) for x0 in x1])
    res = y - f1; res -= res.mean()
    f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])
```

3. Med pakker — BSpline GAM (forelesers eksamen 2025-metode)

```
from ISLP.transforms import BSpline

# 1. Identifiser ikke-lineære variabler
for i, col in enumerate(var_names):
    xi = X[:, i]
    r2_lin = sm.OLS(y, sm.add_constant(xi)).fit().rsquared
    r2_quad = sm.OLS(y, sm.add_constant(
        np.column_stack([xi, xi**2]))).fit().rsquared
    print(f'{col}: delta_R2 = {r2_quad - r2_lin:.3f}')

# 2. Bygg GAM-matrise (x2, x3 ikke-lineære – resten lineære)
bs2 = BSpline(internal_knots=[-1,0,1], degree=3, intercept=False)
X2_sp = bs2.fit_transform(X[:,1].reshape(-1,1))
X_gam = np.column_stack([np.ones(n), X[:,0], X2_sp, X[:,2]])
gam = sm.OLS(y, X_gam).fit()

# 3. MSE og R²
mse_gam = np.mean((y - gam.predict(X_gam))**2)
r2_gam = 1 - mse_gam / np.var(y, ddof=1)
```

4. Annet

- Sammenlign train-MSE(OLS) vs. train-MSE(GAM) — lavere for GAM bekrefter ikke-linearitet.

13. DECISION TREES (REGRESJON & KLASSIFIKASJON)

1. Info

- Rekursiv binær splitting: finn X_j og s som minimerer $RSS(R1) + RSS(R2)$. Grådige valg — globalt suboptimalt.
- Prediksjon: ■ i bladet (regresjon), majoritetsklasse (klassifikasjon).
- Gini: $G = \sum p \blacksquare mk(1-p \blacksquare mk)$. Ren node $\rightarrow G=0$. Brukes som splittingskriterie ved klassifikasjon.
- Overfitting: ubegrenset tre $\rightarrow R^2=1$. Begrens: `min_samples_leaf`, `max_depth`, eller pruning.
- Pruning: $\min RSS + \alpha \cdot |T|$ over subtrær. Velg α med CV (`cost_complexity_pruning_path`).

2. Manuelt — første split (forelesers Tutorial 4)

```
from scipy.optimize import minimize_scalar

def RSS_split(s, x, y):
    R1 = y[x < s]; R2 = y[x >= s]
    if len(R1)==0 or len(R2)==0: return np.inf
    return ((R1-R1.mean())**2).sum() + ((R2-R2.mean())**2).sum()

# Sammenlign Years og Hits:
for col in ['Years', 'Hits']:
    x = X_train[col].values
    res = minimize_scalar(RSS_split, bounds=(x.min(), x.max()),
                          args=(x, y_train), method='bounded')
    print(f'{col}: s={res.x:.2f}, RSS={res.fun:.4f}')

# R² manuelt:
TSS = ((y - y.mean())**2).sum()
RSS = ((y - tree.predict(X))**2).sum()
R2 = (TSS - RSS) / TSS
```

3. Med pakker (sklearn — forelesers metode)

```
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier, plot_tree
from sklearn.model_selection import GridSearchCV

# Regresjon
tree = DecisionTreeRegressor(min_samples_leaf=15, max_depth=3, random_state=0)
tree.fit(X_train, y_train)
plt.figure(figsize=(12,4))
plot_tree(tree, feature_names=X_train.columns, filled=False, fontsize=8)

# Pruning
path = tree.cost_complexity_pruning_path(X_train, y_train)
grid = GridSearchCV(DecisionTreeRegressor(random_state=0),
                    {'ccp_alpha': path.ccp_alphas}, cv=5)
grid.fit(X_train, y_train)
best = grid.best_estimator_

# Klassifikasjon
clf = DecisionTreeClassifier(min_samples_leaf=10, max_depth=4, random_state=0)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
prob = clf.predict_proba(X_test)[:,:1]
```

4. Annet — MAD

```
def MAD(y, pred): return np.mean(np.abs(y - pred))
print('MAD:', MAD(y_test, tree.predict(X_test)))
```

14. BAGGING / RANDOM FOREST / BOOSTING / BART

1. Info

Metode	Trær	Strategi	Nøkkelparameter
Bagging	Mange	Bootstrap + snitt alle pred.	<code>n_estimators</code> , <code>max_features=p</code>
Random Forest	Mange	Bootstrap + m pred./split	<code>max_features=m</code> ($\sqrt{p}$ klass., $p/3$ reg.)
Boosting	Mange sekvensielle	Tren på residualer	<code>n_estimators</code> , <code>learning_rate</code> , <code>max_depth</code>
BART	Mange	Bayesiansk tresum + posterior	<code>n_trees</code> (ISLP.bart)

- OOB error: obs ikke trukket i bootstrap (~37%) = gratis valideringssett.
- Variable importance: gjennomsnittlig RSS-reduksjon per split. `rf.feature_importances_`

2. Manuelt

```
# Bagging for hånd (konsept):
preds = []
for b in range(B):
    idx = np.random.choice(n, n, replace=True)
    tree_b = DecisionTreeRegressor()
    tree_b.fit(X_train.iloc[idx], y_train.iloc[idx])
    preds.append(tree_b.predict(X_test))
y_bag = np.mean(preds, axis=0)
```

3. Med pakker (sklearn — forelesers ames.py / Tutorial 4)

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.ensemble import RandomForestClassifier

# Bagging (max_features = p = alle prediktorer)
bag = RandomForestRegressor(n_estimators=500,
                           max_features=X_train.shape[1], random_state=123)
bag.fit(X_train, y_train)

# Random Forest
rf = RandomForestRegressor(n_estimators=500, max_features=6,
                          random_state=123)
rf.fit(X_train, y_train)
print(MAD(y_test, rf.predict(X_test)))

# Variable importance
pd.Series(rf.feature_importances_,
          index=X_train.columns).sort_values().plot.barh()

# Gradient Boosting
boost = GradientBoostingRegressor(n_estimators=1000,
```

```

        learning_rate=0.01, max_depth=2, random_state=123)
boost.fit(X_train, y_train)

# BART
from ISLP.bart import BART
bart = BART(n_trees=200, random_state=0)
bart.fit(X_train, y_train)

```

4. Annet — klassifikasjon

```

rfc = RandomForestClassifier(n_estimators=500, max_features='sqrt',
                             random_state=123)

rfc.fit(X_train, y_train)
prob = rfc.predict_proba(X_test)[: ,1]
pred = (prob > 0.5).astype(int)
pd.crosstab(y_test, pred, rownames=['Actual'], colnames=['Pred'])

```

15. NEURAL NETWORKS (MLPREGRESSOR / MLPCLASSIFIER)

1. Info

- Pensum: kun MLPRegressor / MLPClassifier fra sklearn.neural_network. Ikke PyTorch, CNN, RNN.
- hidden_layer_sizes=(15,) = ett skjult lag, 15 nevroner. (15,8) = to lag.
- Aktivasjon: ReLU: $g(z)=\max(0,z)$. Sigmoid: $g(z)=e^z/(1+e^z)$.
- Optimering: gradient descent (ikke-konveks $\rightarrow$ lokal minimum). Alltid StandardScaler!

2. Manuelt

```

# Forward pass for ett skjult lag (konsept):
# Z1 = X @ W1 + b1; A1 = relu(Z1)
# Z2 = A1 @ W2 + b2; output = Z2 (reg.) eller sigmoid(Z2) (klass.)
# Ikke nødvendig å implementere – bruk sklearn

```

3. Med pakker (sklearn — forelesers ames.py)

```

from sklearn.neural_network import MLPRegressor, MLPClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_train) # fit KUN på train!
X_te_s = scaler.transform(X_test)

# Enkelt lag
nn = MLPRegressor(hidden_layer_sizes=(15,), activation='relu',
                  max_iter=100000, random_state=0)
nn.fit(X_tr_s, y_train)
print(MAD(nn.predict(X_te_s), y_test))

# Dypt nettverk
dnn = MLPRegressor(hidden_layer_sizes=(15,8), activation='relu',
                  max_iter=100000, random_state=0)
dnn.fit(X_tr_s, y_train)

# Klassifikasjon
mlpc = MLPClassifier(hidden_layer_sizes=(50,25), activation='relu',
                    max_iter=1000, random_state=0)
mlpc.fit(X_tr_s, y_train)
prob = mlpc.predict_proba(X_te_s)[: ,1]

```

16. SVM (SUPPORT VECTOR MACHINES)

1. Info

- Maximal margin classifier: finn hyperplan som maksimerer margin M mellom klasser.
- Support vector classifier (SVC): tillat feilklassifiseringer — styrt av C (cost).
- C liten: smal margin, lavere varians. C stor: bred margin, mer fleksibel, høyere varians.
- Kernel SVM: mapper data til høyere dimensjon. Polynom: $(1+x \cdot x')^d$. RBF: $\exp(-\gamma \|x-x'\|^2)$.
- RBF γ liten = glatt grense. γ stor = kompleks/overfit. Velg C og γ med GridSearchCV.

2. Manuelt

- Ingen enkel manuell implementasjon — SVM-optimering er kvadratisk programmering. Bruk sklearn.

3. Med pakker (sklearn — forelesers ex97.py)

```

from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

# Lineær SVC
param_lin = {'C': [0.01, 0.1, 1, 10, 100, 1000]}
tune_lin = GridSearchCV(SVC(kernel='linear'), param_lin, cv=5,
                       scoring='accuracy')

tune_lin.fit(X, y)
best_lin = tune_lin.best_estimator_

# Polynom-kernel
param_poly = {'C': [0.01, 1, 100, 1000], 'degree': [1, 2, 3, 4]}
tune_poly = GridSearchCV(SVC(kernel='poly'), param_poly, cv=5)
tune_poly.fit(X, y)

# RBF-kernel
param_rbf = {'C': [0.01, 1, 100, 1000], 'gamma': [0.5, 1, 2, 4]}
tune_rbf = GridSearchCV(SVC(kernel='rbf'), param_rbf, cv=5)
tune_rbf.fit(X, y)

# Prediksjon og evaluering
from sklearn.metrics import confusion_matrix
pred = tune_rbf.best_estimator_.predict(X_test)
print(confusion_matrix(y_test, pred))

```

4. Annet — visualisering

```
from ISLP.svm import plot as plot_svm
fig, ax = plt.subplots(1,2)
plot_svm(X, y, best_lin, ax=ax[0], features=(0,1))
plot_svm(X, y, best_rbf, ax=ax[1], features=(0,1))
```

17. CLUSTERING (UNSUPERVISED)

1. Info

- K-means: velg K, tilordne til nærmeste sentroid, oppdater sentroid=gjennomsnitt, gjenta til stabilitet.
- Mål K-means: min within-cluster variation = $\sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$.
- Hierarkisk: agglomerativ — start n clusters, slå sammen to nærmeste → dendrogram.
- Linkage: Complete=max avstand (kompakt), Single=min (kjedende), Average, Ward=min total varians.
- Dendrogram: y-akse = avstand ved sammenslåing. Stort hopp = naturlig skille. Horisontalt kutt → K clusters.
- Alltid standardiser X før clustering (avstandsbasert)!

2. Manuelt — K-means

```
# Enkel K-means fra scratch (2D):
def kmeans(X, K, iters=100):
    idx = np.random.choice(len(X), K, replace=False)
    centers = X[idx].copy()
    for _ in range(iters):
        dists = np.array([np.sum((X-c)**2, axis=1) for c in centers])
        labels = np.argmin(dists, axis=0)
        centers = np.array([X[labels==k].mean(axis=0) for k in range(K)])
    return labels, centers
```

3. Med pakker (sklearn — forelesers Tutorial 4)

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, cut_tree
from ISLP.cluster import compute_linkage

# K-means
scaler = StandardScaler()
X_s = scaler.fit_transform(X)
km = KMeans(n_clusters=3, random_state=1, n_init=10)
km.fit(X_s)
pd.crosstab(km.labels_, y_true)

# Hierarkisk
hc = AgglomerativeClustering(distance_threshold=0, n_clusters=None,
                             linkage='complete')
hc.fit(X_s)
lm = compute_linkage(hc)
dendrogram(lm, no_labels=True)
clusters = cut_tree(lm, n_clusters=3).reshape(-1)
pd.crosstab(clusters, y_true)
```

18. QUICK REFERENCE

1. Standard imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import sklearn.linear_model as skl
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPRegressor, MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, cut_tree
from scipy.optimize import minimize, minimize_scalar
from ISLP import load_data
from ISLP.models import ModelSpec as MS, summarize
from ISLP.transforms import BSpline
from ISLP.cluster import compute_linkage
from ISLP.bart import BART
```

2. Metrikk

Metrikk	Formel	Python
MSE	$\text{mean}(y - \hat{y})^2$	<code>np.mean((y-pred)**2)</code>
RMSE	$\sqrt{\text{MSE}}$	<code>np.sqrt(np.mean(...))</code>
MAD/MAE	$\text{mean}(y - \hat{y})$	<code>np.mean(np.abs(y-pred))</code>
R ²	$1 - \text{RSS} / \text{TSS}$	<code>model.score(X,y)</code>
Accuracy	$(\text{TP} + \text{TN}) / n$	<code>np.mean(pred==y)</code>

3. Confusion matrix

```
cm = pd.crosstab(y_true.values, pred,
                 rownames=['Actual'], colnames=['Predicted'])
#           Pred 0    Pred 1
# Actual 0    TN     FP
# Actual 1    FN     TP
# Accuracy=(TP+TN)/n  Sensitivity=TP/(TP+FN)  Specificity=TN/(TN+FP)
```

4. Eksamensmønster 2024 & 2025

- 2025 oppg.1: Ridge (g()-funksjon, demeaning) → LOO-CV for α → Bootstrap (varians, persentil-KI) → GAM (BSpline, delta-R²).
- 2024 oppg.1: KNN-regresjon fra scratch → LOO for K → backfitting-GAM med KNN-smoother.
- 2024 oppg.2: EDA (scatter/boxplot/krysstabell) → logistisk reg. → klass.tre → RF m/ variable importance.

- Alltid: forklar valg av train/test/fullt datasett. Juster terskel ved ubalanserte klasser.
- Deskriptiv stats: scatter (kon./kon.), boxplot (kon./kat. og kat./kon.), krysstabell (kat./kat.).